SELENIUM PYTHON AUTOMATION
MODULE 4 DISCUSSION WORKBOOK

Robot Framework — Concepts, Working Knowledge and Framework Practices

Module	Module 4 — Robot Framework
Curriculum basis	CoE-TOC-PYTHON_AUTOMATION.xlsx — Robot Framework section
Discussion scope	Robot Framework syntax, keywords, variables, data-driven testing, libraries, assertions, setup,
Primary web library	SeleniumLibrary
Evidence status	Discussion/knowledge report; no execution results are claimed in this module

Module purpose

The course curriculum introduces Robot Framework as a separate automation framework after the Selenium, PyTest and Behave material. The Module 4 discussion therefore focuses on understanding how Robot Framework test data is organized, how keywords and libraries provide executable actions, and how framework features support reusable, readable and maintainable automation.

Curriculum topics identified in the course material

- Introduction to Robot Framework and environment/package setup.
- Basic syntax, keywords and browser automation using SeleniumLibrary.
- Variables and data-driven test cases.
- Custom keywords, Python libraries and the BuiltIn library.
- Assertions and verification.
- Test setup and teardown.
- Tags, command-line execution and execution options.
- Reports and logs.
- RIDE, PyCharm/command-window execution, batch/task scheduling and Jenkins.
- User-defined keywords, resource files, Gherkin readability and an overview of Page Objects.

1. Robot Framework Syntax and Keyword-Driven Automation

Basic structure

Robot Framework test data is organized into sections. The commonly used sections include Settings, Variables, Test Cases and Keywords. Test cases are built by calling available keywords. Keywords may come from imported libraries, resource files or user-defined keyword sections. This creates a keyword-driven style in which the test describes actions rather than embedding Python control logic directly in every test step.

Typical web automation flow

Stage	Conceptual Robot Framework action
Setup	Import SeleniumLibrary and define required variables/settings.
Launch	Open the browser and navigate to the target URL.
Interaction	Use keywords such as Input Text and Click Element/Click Button.
Verification	Use assertion/verification keywords to check expected page state.
Cleanup	Close the browser through a teardown keyword.

SeleniumLibrary

The curriculum specifically points to browser automation using Selenium Library. SeleniumLibrary is a Robot Framework web-testing library that uses Selenium internally. It exposes browser-oriented keywords, allowing actions such as opening a browser, entering text and interacting with page elements to be represented as readable Robot Framework keywords.

Conceptual example:

```
*** Settings ***
Library SeleniumLibrary

*** Test Cases ***
Login Test
    Open Browser https://example.com chrome
    Input Text username testuser
    Click Button Login
```

The example is illustrative knowledge, not execution evidence for this module.

Working knowledge gained

• Robot Framework separates test intent from the implementation of lower-level keywords.

• A keyword can represent a browser action, assertion, setup operation or reusable business-level action.

• SeleniumLibrary provides the browser interaction layer while Robot Framework provides the test-data and execution structure.

2. Variables, Data-Driven Testing and Reusable Keywords

Variables

Robot Framework variables allow values to be reused rather than repeatedly hard-coded in test cases. The framework supports scalar variables such as ${USER}, list variables such as @{USERS}, dictionary variables such as &{CREDENTIALS}, and environment variables using %{}. Variables can be used as keyword arguments and in settings.

Data-driven automation

The curriculum asks for test cases that store values such as usernames and passwords in variables and for data-driven tests that execute the same set of actions over multiple data sets. The important design idea is separating test data from the repeated workflow. This makes it possible to reuse the same automation logic while changing the input values.

Concept	Purpose
Scalar variable	Store one value such as a URL, username or password.
List variable	Represent a collection of values for repeated processing.
Dictionary variable	Represent named key/value data.
External data	Keep test data separate from the automation workflow where appropriate.
Reusable keyword	Encapsulate a sequence of lower-level keywords into a meaningful action.

Custom keywords and libraries

The curriculum includes creating a custom Robot Framework keyword using Python and using the BuiltIn library for common operations such as string manipulation or calculations. User-defined keywords provide a higher-level abstraction: instead of repeating several low-level browser actions, a test can call one meaningful keyword.

This is closely related to the framework design learned in earlier modules: reusable functions/classes in Python and page objects in Selenium have the same broad engineering goal of reducing duplication and separating responsibilities, although Robot Framework expresses the abstraction primarily through keywords and resource files.

Working knowledge gained

- Test data can be separated from workflow logic.
- User-defined keywords improve readability and reduce repeated action sequences.
- Resource files can centralize reusable keywords and variables for multiple suites.

3. Assertions, Setup/Teardown, Tags and Execution

Assertions and verification

The curriculum requires verification of expected outcomes, including an example using Should Be Equal As Strings for comparing actual and expected values. In web automation, the same principle applies to checking that a page, element, text value or application state matches the expected result.

Setup and teardown

Setup and teardown establish a repeatable test lifecycle. A setup prepares the environment before a test, while teardown performs cleanup after execution. Robot Framework supports setup and teardown at suite, test-case and user-keyword levels. Test teardowns are executed even when a test fails, making them suitable for cleanup.

Tags and selective execution

The curriculum introduces tags for classifying tests and running selected tests. Robot Framework supports including or excluding tests through command-line options such as --include and --exclude. Tags therefore provide a simple mechanism for grouping smoke, regression, login or other logical test categories.

Execution options covered by the curriculum

• Run from RIDE, PyCharm or a command window.

• Run a single test case or multiple suites.

• Use advanced command-line execution options.

• Run or schedule automation through Jenkins.

• The curriculum also mentions Sauce Labs browser execution and task/batch scheduling.

Reports and logs

Robot Framework produces execution results that can be used to inspect test outcomes. The curriculum specifically requires generating an HTML report containing test-case results and using the Log keyword to add custom diagnostic messages. The reporting layer is therefore part of the automation workflow rather than an afterthought.

Working knowledge gained

• Test lifecycle management is essential for repeatable automation.

• Tags make larger suites easier to select and organize.

• Logs and reports provide execution visibility and support debugging.

4. Readability, Resource Files, Gherkin and Page Objects

Making scripts more readable

The curriculum explicitly contrasts procedural and Gherkin styles and introduces user-defined keywords as a way to make test cases easier to understand. A readable Robot Framework suite should express what the test is doing at an appropriate abstraction level while moving implementation details into reusable keywords.

Resource files

Moving keywords into resource files allows multiple test suites to reuse the same implementation. This supports a layered framework in which test cases contain high-level intent and resource files contain reusable actions, variables and supporting definitions.

Gherkin-style readability

The curriculum includes increasing readability using Gherkin. The goal is to make test intent understandable in a behaviour-oriented form. This connects conceptually with the Behave work completed in Module 3, where Given/When/Then steps were used to express behaviour while Python step definitions provided the implementation.

Page Object overview

The curriculum also introduces Page Objects. In a Robot Framework context, the same separation principle can be implemented through reusable keywords and resource organization: page-specific interaction details are kept away from the high-level test flow. The course material identifies this as an overview rather than a separate completed Module 4 implementation.

Module 4 learning summary

• Robot Framework uses readable, keyword-driven test data instead of requiring every test step to be written in Python.

• SeleniumLibrary provides Selenium-backed browser keywords for web automation.

• Variables and data-driven approaches support reuse of the same workflow with different inputs.

• User-defined keywords and resource files provide framework-level reuse and maintainability.

• Setup, teardown, tags, command-line execution, logs and HTML reports provide the operational test framework.

• Gherkin and Page Object concepts improve separation of test intent from implementation details.

Result and scope note

Module 4 is documented here as a discussion and knowledge workbook based on the course curriculum and supporting official Robot Framework documentation. No Module 4 execution result, pass/fail result or completed automation implementation is claimed in this report because corresponding execution evidence was not available.

References

1. CoE-TOC-PYTHON_AUTOMATION.xlsx — Python Automation ToC, Robot Framework section (items 3.1–3.3).

2. Robot Framework User Guide — sections covering test cases, variables, tags, setup/teardown and execution.

3. Robot Framework Documentation — Variables and SeleniumLibrary documentation.